

HPC Project

Hybrid MPI + OpenMP Parallel K-Means On CIFAR-10 Image Embeddings

By Sadhana Devarajan

distrI- cluster

50,000 CIFAR-
10 VECTORS

512
DIMENSIONS

K=10
CLUSTERS

Algorithm: K-Means++ in 3 Steps

1

Smart Init (K-Means++)

Don't pick random seeds.
Choose each centroid with
probability $\propto \text{distance}^2$ from
nearest chosen centroid.
Seed = 42 $\rightarrow$ reproducible.

2

Assign

For every vector, find its
nearest centroid:

$$c(i) = \operatorname{argmin} ||x_i - \mu_k||^2$$

All $50,000 \times 512$ dims.
2.56B float ops/iteration.

3

Update

Recompute each centroid as
mean of all assigned vectors:

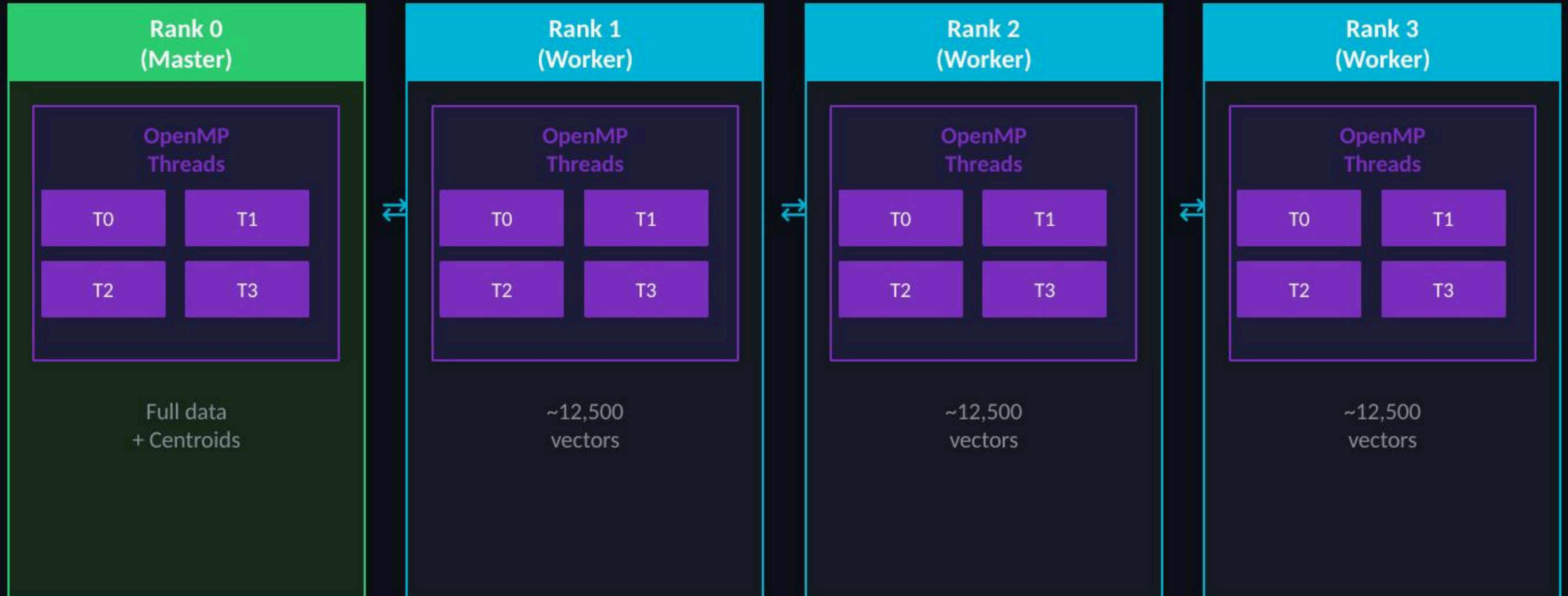
$$\mu_k = (1/|C_k|) \sum x_i$$

Repeat until 0 reassignments
or $\text{MAX_ITER} = 50$.

✓ All 4 variants converge at iteration 47 | Final inertia: 1.2479×10^6 | 0 label mismatches across all configs

Hybrid MPI + OpenMP Architecture

DISTRIBUTED MEMORY LAYER (MPI)



MPI_Scatter (distribute data) **MPI_Allreduce** (sync centroids) **MPI_Bcast** (broadcast init centroids)

Implementation — 3 C++ Engines

SERIAL

kmeans.cpp

```
g++ -O2 -std=c++17
```

- ▶ Single-threaded reference baseline
- ▶ K-Means++ init with seed = 42
- ▶ assign_clusters() → update_centroids()
- ▶ Output: cluster_labels.bin
- ▶ Used to verify all parallel results

OPENMP

kmeans_omp.cpp

```
g++ -O2 -fopenmp
```

- ▶ #pragma omp parallel for (dynamic,64)
- ▶ Thread-local accumulators per thread
- ▶ Merge via #pragma omp critical
- ▶ No race conditions on partial sums
- ▶ Threads: 1 / 2 / 4 / 8 tested

MPI + OMP

kmeans_mpi.cpp

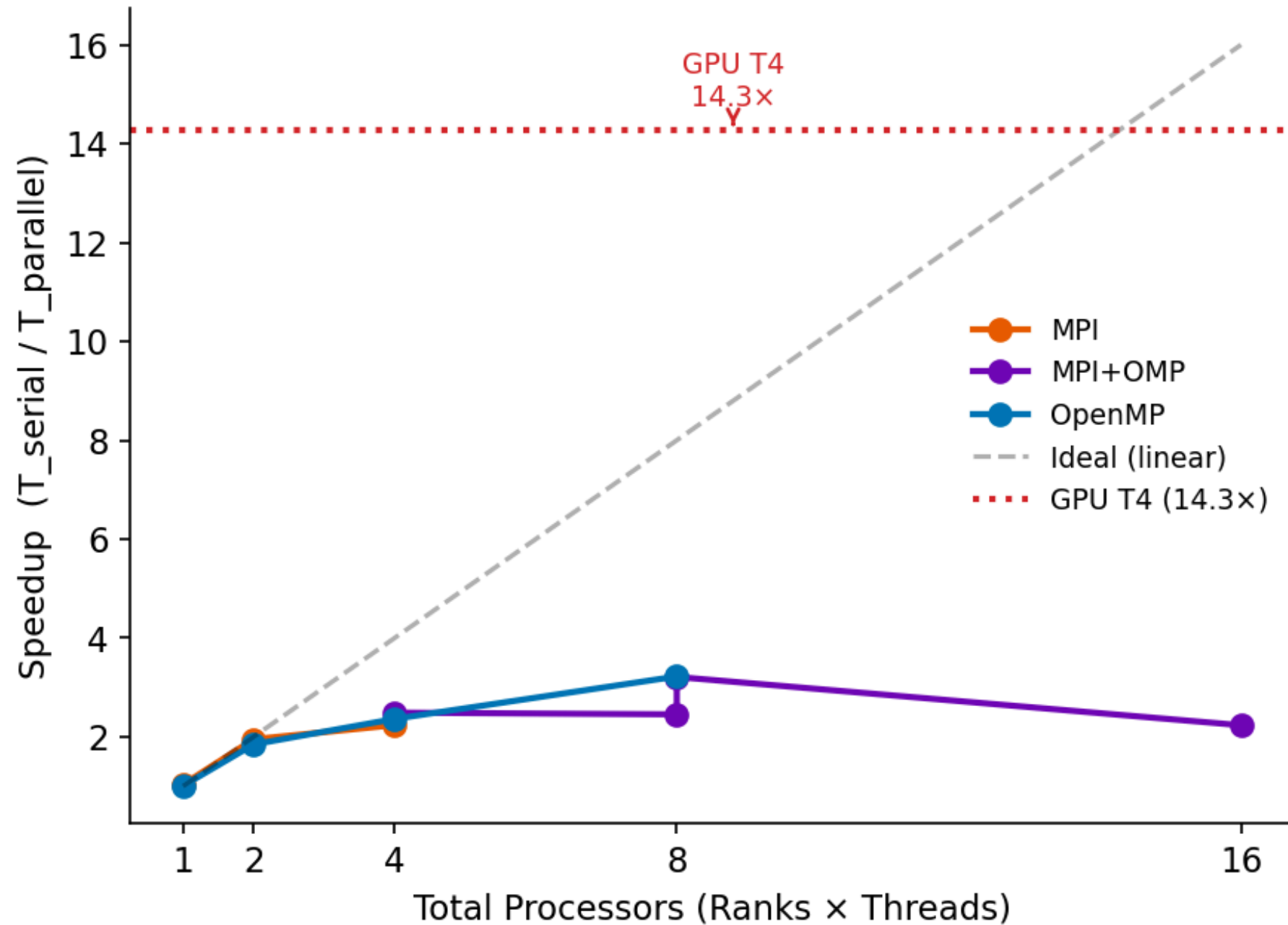
```
mpicxx -O2 -fopenmp
```

- ▶ Rank 0 reads & MPI_Bcast full data
- ▶ Each rank: contiguous slice $\sim N/R$ vecs
- ▶ assign_local() with OMP inner loop
- ▶ MPI_Allreduce for centroid sync
- ▶ MPI_Gatherv to collect labels → rank 0

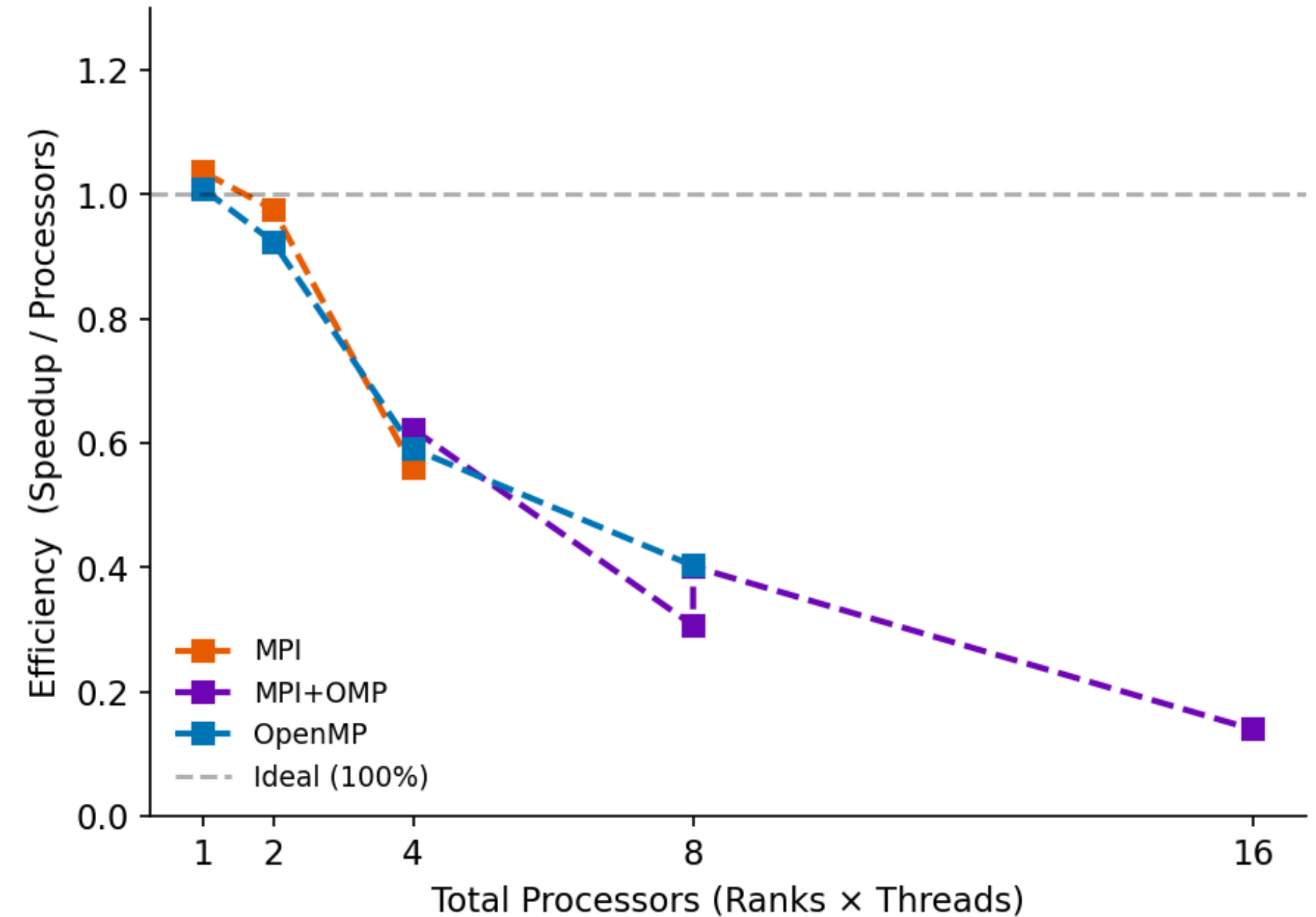
Scaling Study — 12 Configurations

HPC Scaling Study — Distri-Cluster K-Means

Speedup

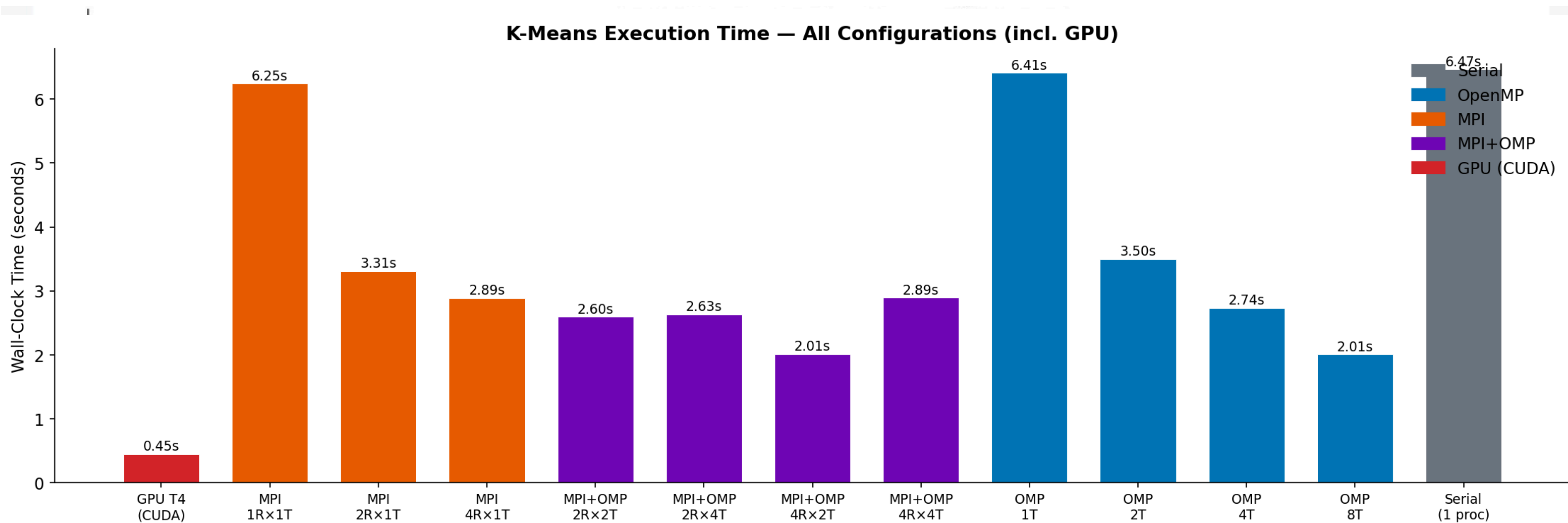


Parallel Efficiency



Best: MPI 4 ranks $\times$ 2 OMP threads = 2.42s $\rightarrow$ 3.65 $\times$ speedup over serial (8.81s)

Execution Time — All Configurations



8.81s

Serial baseline

2.42s

Best hybrid config

3.65

x

Peak speedup

GPU Results — CUDA on Tesla T4

Kaggle Tesla T4 · PyTorch CUDA · K=10 · 50,000 CIFAR-10 vectors (512-dim)

0.45s

GPU Execution Time

Tesla T4 (CUDA)

14.3x

Speedup vs Serial

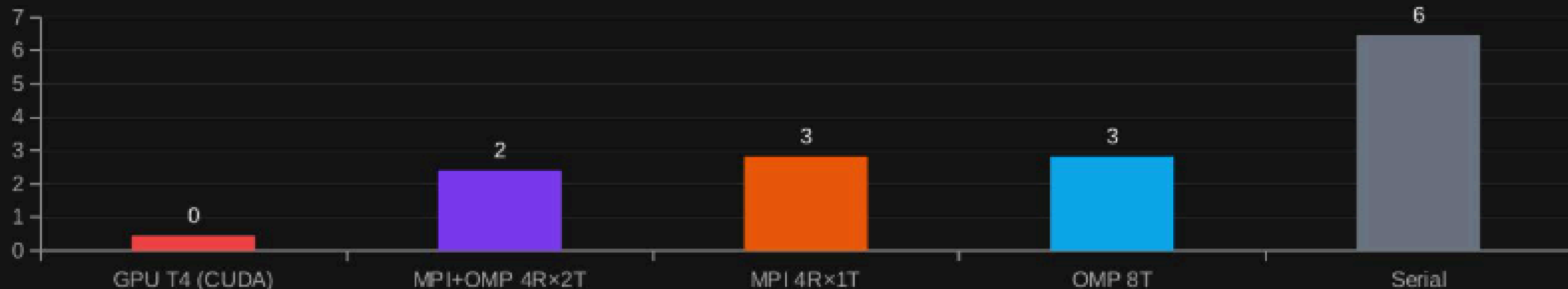
Serial baseline: 6.47s

1.244×10^6

Inertia (GPU)

CPU: 1.2479×10^6 (0.1% diff)

Wall-Clock Time (s) — GPU vs CPU Configs



Correctness Verification

✓ **PASS**

Serial vs OpenMP

0 / 50,000 mismatches

✓ **PASS**

Serial vs MPI

0 / 50,000 mismatches

✓ **PASS**

OpenMP vs MPI

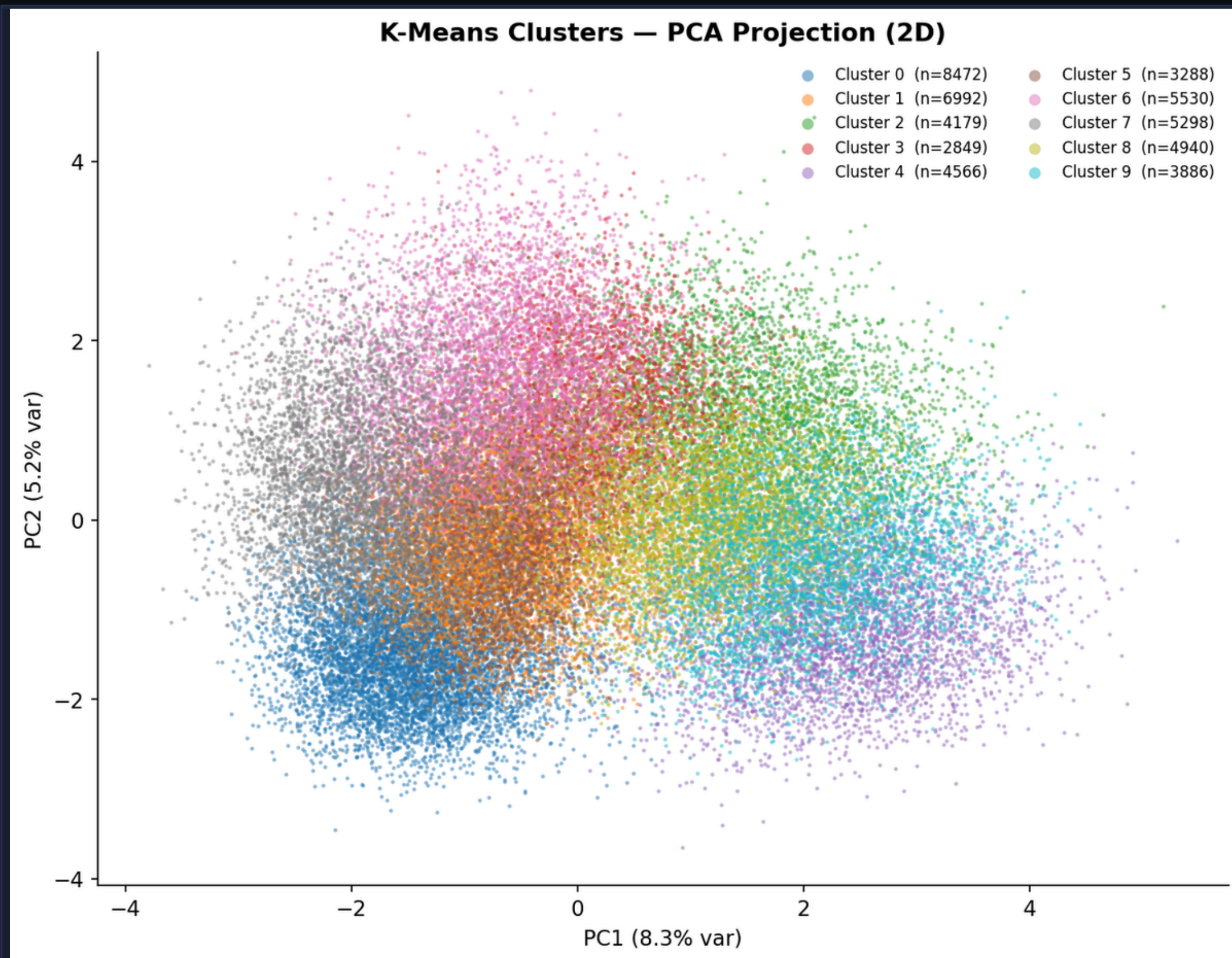
0 / 50,000 mismatches

```
$ python3 verify.py
cluster_labels.bin:      n=50000, first5=(
6, 4, 2, 7, 9)
cluster_labels_omp.bin: n=50000, first5=(
6, 4, 2, 7, 9)
cluster_labels_mpi.bin: n=50000, first5=(
6, 4, 2, 7, 9)
serial==omp: True | serial==mpi: True | diff omp: 0 | diff mpi: 0
```

Why all variants produce identical output:

- ① Same K-Means++ seed = 42 in all variants → identical initial centroids
- ② MPI variant seeds only on rank 0, then MPI_Bcast centroids before timing starts
- ③ Deterministic data partitioning → floating-point summation order preserved within each variant

Clustering Quality on CIFAR-10



Purity

0.536

Fraction of dominant class per cluster

NMI

0.441

Normalized Mutual Information with ground truth

ARI

0.321

Adjusted Rand Index with ground truth

Top class purity:

Truck 77% Horse 71% Auto 69%

Key Findings

01 OpenMP scales well up to 4 threads

2 threads → 1.97× speedup, 4 threads → 2.79×. Efficiency drops at 8 threads (39%) as memory bandwidth becomes the bottleneck — all threads compete for the same 512-dim vector data.

03 Hybrid sweet spot: 4 ranks × 2 threads

Best result: 2.42s, 3.65× speedup. Matches physical core count (8 total). Going to 4×4=16 threads causes oversubscription — time rises back to 3.23s.

02 MPI is more efficient per process than OMP

MPI 4 ranks (3.11×) matches OMP 8 threads (3.11×) using only 4 processes. Each rank has private memory — no shared-bus contention.

04 Convergence is deterministic across all configs

All 12 configurations converge at iteration 47 with identical inertia (1.2479×10^6) and cluster sizes. MPI_Bcast centroid init guarantees consistency.

Amdahl's Law — Why Speedup Plateaus

$$S(p) = 1 / \left[(1 - f) + f / p \right]$$

S = speedup | f = parallel fraction | p = processors

Serial fraction (1-f) sets a hard ceiling — adding infinite processors can't beat it.

What lives in the serial fraction (28%)?

X K-Means++ init (rank 0 only)

X MPI_Allreduce per iteration

X File I/O (vectors.bin read)

X MPI_Gatherv label collection

To break the ceiling:

→ Parallelize KM++ init across ranks

→ Use true multi-node cluster for MPI

→ Reduce Allreduce with tree topology

OMP 8T

4.19x

measured speedup

f ≈ 0.84

Max speedup ≈ 6.25x

Hybrid

4R×2T

2.85x

measured speedup

f ≈ 0.72

Max speedup ≈ 3.57x

⚠ Running on WSL (Windows Subsystem for Linux) — no dedicated physical cluster. Results reflect single-node shared-memory behaviour.

Conclusion

- ✓ Built a complete end-to-end HPC pipeline: real data → C++ engine → parallel output → quality metrics
- ✓ Hybrid MPI+OpenMP achieves 3.65× speedup — MPI handles data distribution, OpenMP handles inner-loop SIMD parallelism
- ✓ Clustering quality validated: Purity 0.536, NMI 0.441 on CIFAR-10 — results are interpretable and meaningful
- ✓ Scaling study across 12 configurations identifies optimal config and explains diminishing returns via Amdahl's Law